

How Robots Might Reproduce: A *Futurama* Inspired Recombination Simulation Using Lambda Calculus

Cooper Koers

May 2025

Code Availability: github.com/cooperkoers/LambdaForge

This assignment represents my own work in accordance with University regulations.

1 Introduction: Robots, Reproduction, and the Allure of Science Fiction

Beginning from the earliest depictions of mechanical humanoids, robots have captivated audiences throughout the genre of science fiction. Whether depicted as mindless workers, unrelenting logicians, or human-like partners in crime, they have enthralled audiences for decades. From *2001: A Space Odyssey*'s HAL 9000 to *Star Wars*' C-3PO, robots have provided viewers with a mirror into the aspects of humanity that make us who we are [11] [12]. By artificially mimicking humanity, we are offered a new insight into the very processes and features that define life. How do we create objects that move, behave, and interact with the universe in fashions similar to us? This is the ultimate attraction to the trope [14].

In the realm of popular science fiction, few shows have attempted to answer these questions of robotic anthropomorphism with such charm and absurdism as Matt Groening's animated series *Futurama* [10]. The robots in the series largely mimic human behavior – having jobs, personalities, opinions, and often comically: sex lives. Robots engage in similar courting and mating behavior to humans, where robots are often seen flirting, dating, and even marrying each other. Bender, the show's robotic girder-bending antihero, is shown in the series to even father a child with his company's soda machine Bev in the Season 7 episode “The Bots and the Bees” [9]. Through the humorous plot of following his learning curve in becoming a parent to a child that does not resemble his abilities, Bender's experience is meant to offer a mirror to the audience covering the human concept of parenthood.

Yet, this offers a strange, but intriguing question: What does it mean for robots to reproduce? Not in a metaphorical sense, but rather in a literal one. In biology, reproduction is the sharing of genetic material driven by evolutionary concepts of selection and variation [3]. Is there a way to abstract this process to the lens of robots and their algorithmic cognitive architectures?

In this paper, I seek to explore this possibility through a computational and mathe-

mathematical lens by utilizing Lambda calculus: a system created by Alonzo Church in the 1930s to elementalize computation by considering merely function abstraction and application to describe all logical processes [4]. While this may seem like an outlandish concept in comparison to Bender’s hilarious mean-spirited story arc, Lambda calculus provides reproduction between robotic entities an elegant and fertile platform to explore. Instead of DNA, the genome of a robot is a Lambda expression: the mathematical structure that dictates its logic that defines its very being. Considering this, reproduction could be realized as recombining and mutating these expressions to create novel offspring. Given the ability to produce offspring in this manner, the system of artificial reproduction lies not in circuitry or gears, but in pure logic.

In this project, I have designed a software titled LambdaForge that simulates these logical reproductive processes in order to further explore the concepts of robotic reproduction in *Futurama* that mirror those of organic life. While not necessarily geared to building the logical infrastructure required to pull off large-scale robotic reproduction, what LambdaForge does offer is a framework to explore the conceptual notion that robots may be able to reproduce while mimicking the biological concepts of novelty, inheritance, and evolution not from blueprints and predefined processes, but rather from function and recursion.

2 Lambda Calculus: A Friendly Primer into Logical Reproduction

2.1 What Is Lambda Calculus?

Lambda calculus is a mathematical language invented by the logician Alonzo Church in the 1930s that decomposes all computation to three concepts: variables, function definitions, and function applications [4]. Traditional logic structures of computation such as loops, if-statements, and even numbers are derived from pure logic. From these three con-

cepts, every process can be built: addition, multiplication, and all complex algorithms. Bio-computationally, Lambda expressions are analogous to the DNA backbone of code. In a similar fashion to how life emerges from the functionality derived from the code stored in genomes, all computational power can be expressed with the backbone that Lambda expressions provide [4]. For purposes of this study, it is an excellent framework to explore algorithmic reproduction.

2.2 Why Lambda Calculus?

A beautifully simple notion in biology is the programming language that the four nucleic acids provide to encode biological processes on a global scale between all life [15]. Genomics allows for a universality in biological analysis, where encoding and decoding is consistent such that all biological logic can be described using the four-letter alphabet of DNA. Molecular structures, complex regulatory networks, and even reproduction itself can be described using only these four molecules, where the molecules encoded for by DNA – all varieties of RNAs and proteins – act as an abstraction of the genetic code [7]. The actions and processes in which RNA and protein molecules participate in are themselves a decoding of sorts of the DNA code: including reproduction. Considering that reproduction is a decoding and abstraction of DNA, it became necessary that I find a suitable framework to encode computation that is Turing complete – meaning that it can perform any computational problem given enough memory and time. Lambda calculus fulfills these needs [13].

2.3 The Three Constructs

Lambda calculus may only have three core constructs, but combined they are powerful enough to model all computation.

1. Variables

Variables are abstract names – placeholder strings that stand in for representation. Simply put, they are variables in the traditional mathematical sense, with identical notation such

as x , y , or z .

2. Abstractions

Abstractions are how one defines a function. The structure is similar to traditional mathematics. Rather than say $f(x) = x + 2$, a Lambda abstraction for this function would look like $\lambda x.x + 2$, reading as "given input x , return $x + 2$ ". The role of the character λ is to declare that a function is being created.

3. Applications

Applications additionally mirror traditional function application. Rather than solving $f(x) = x + 2$ for $x = 4$ as $f(4) = 4 + 2 = 6$, Lambda calculus function applications follow the following notation: $(\lambda x.x + 2)(4)$, reading as "apply the number 4 to the value x within the function that is defined by adding two to x ".

This is *nearly* Lambda calculus! $x + 2$ assumes that we have defined what both numbers and addition are, but this has not been done yet in the expression. In pure Lambda calculus, each structure of a mathematical statement has to be built from scratch using only abstraction and application of variables. To do this, a logical way of representing commonly used notation is required.

2.4 Building Algorithms using Lambda Calculus

To construct the basic Lambda expressions for usable logic or mathematical algorithms, there is a need to define building blocks that allow for higher level abstractions such as the numbers or arithmetic operators. To achieve this, there needs to be a building of logic.

2.4.1 Encoding of Algorithmic Foundations

There have been common notations developed in Lambda calculus to create more readable, higher-level abstractions of processes that use readable language in favor of complex nesting of Lambda expressions. For example, in the explanation above for the three constructs of Lambda calculus, the function $f(x) = x + 2$ served as the example. However, there was

no definition of what 2 or addition is logically. As an example of encoding computation, Alonzo Church created Church Numerals to represent numbers, a system that represents the positive integers as the number of times an arbitrary function is applied to a variable [1].

Example: Church Numerals

Number	Function Definition	Lambda Expression
0	$0 \ f \ x = x$	$\lambda f. \lambda x. x$
1	$1 \ f \ x = f \ x$	$\lambda f. \lambda x. f \ x$
2	$2 \ f \ x = f(f \ x)$	$\lambda f. \lambda x. f(f \ x)$
3	$3 \ f \ x = f(f(f \ x))$	$\lambda f. \lambda x. f(f(f \ x))$
n	$n \ f \ x = f^n(x)$	$\lambda f. \lambda x. f^{on}(x)$

Table 1: Church numerals represented as Lambda expressions

3 From Futurama to Function: The Recombinatorial Reproduction Engine LambdaForge

Given that Lambda expressions offer a means of encoding logic and computation, an opportunity to further mirror organic life is presented by exploring reproduction using this framework. To explore recombination between algorithmic processes, I created a model to simulate a purely stochastic model of recombination and reproduction. The model inputs two Lambda expressions and parses them as a syntax tree. In this parsing, each abstraction, application, and variable from both expressions are stored in a list. Then, an abstraction is chosen at random, and any other object in the list is applied to it. From this, new expressions can be constructed using the logic defined by the parent expressions.

3.1 Lambda Syntax Trees: Selecting Material to Pass On

To visualize these input expressions, I employ the syntax tree: a way to visualize Lambda expressions as a tree. From these diagrams, each logical step is displayed such that the

progression of function abstraction and application is made clear. Below is the syntax tree for the function that adds the number 2 to any integer, denoted with the Lambda expression:

$$ADD2 = \lambda n. \lambda f. \lambda x. f(f(n f x))$$

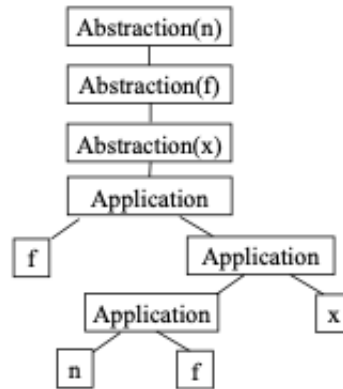


Figure 1: This is the full syntax tree representation of ADD2 using purely Lambda calculus terms.

To the unfamiliar eye, this should look nonsensical – a confusion of random variables applied to each other. Yet, in the chaos there are patterns that arise. Within the expression, there lies a hidden function - the successor function defined as:

$$SUCC = \lambda n. \lambda f. \lambda x. f(n f x)$$

This motif of $f(n f x)$ is seen in ADD2, but wrapped in an outer function wrapper: akin to the concept of the Church Numeral 2. Given this logic, the ADD2 function abstracts to the concept of applying the successor function twice to an argument. This resulting syntax tree of abstraction allows for the visualization of the repeated succession.

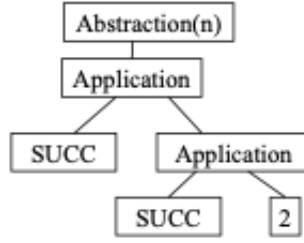


Figure 2: This is the higher-level representation of ADD2 using abstractions for the successor function.

In this visualization, the recombinatorial processes of LambdaForge are revealed. Given an expression, LambdaForge considers each resulting downward sub-tree formed from each point in the syntax tree.

3.2 Forming a New Expression

Reproduction is simulated in LambdaForge considering each sub-tree within the syntax tree format of parent Lambda expressions. From each sub-tree between the parent expressions, one random abstraction sub-tree and one sub-tree of any type is selected. The sub-tree of any type is then applied to the abstraction sub-tree, creating a new function using the features of the logic pool contained between the parent expressions.

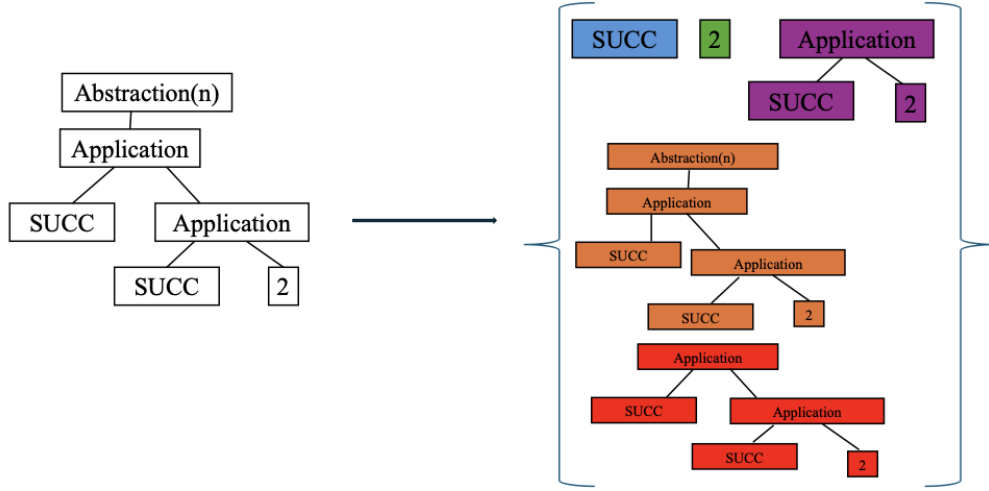


Figure 3: Example of each unique logic sub-tree within a Lambda Expression.

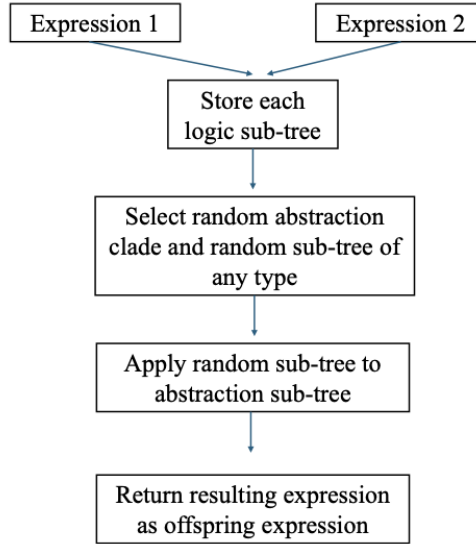


Figure 4: Flow chart of LambdaForge reproduction software.

While the software operates at the level of the Lambda expression, the interface for the user is friendly to both Lambda expression inputs and higher-level abstractions. Therefore, users are able to produce offspring functions by using familiar syntax for algorithm building. Listed below are the supported higher-level abstractions for LambdaForge as of the current release:

Name	Lambda Expression
TRUE	$\lambda t.\lambda f. t$
FALSE	$\lambda t.\lambda f. f$
AND	$\lambda p.\lambda q. p\ q\ p$
OR	$\lambda p.\lambda q. p\ p\ q$
ADD	$\lambda m.\lambda n.\lambda f.\lambda x. m\ f\ (n\ f\ x)$
MULTIPLY	$\lambda m.\lambda n.\lambda f. m\ (n\ f)$
EXPONENTIATE	$\lambda m.\lambda n. n\ m$

Table 2: Lambda calculus expressions for Boolean logic and arithmetic (Church encoding)

4 The Core Concepts: Reproduction, Evolution, Novelty

4.1 Reproduction: Plug and Play Parenthood

In the episode "The Bots and the Bees", viewers are introduced to the mechanism that underlies reproduction between robots in the *Futurama* universe [9]. To help meet factory demands, robots have designed with the ability to reproduce. Copulation is described as the "uploading of a male binary female into the female hard drive", where manufacturing specification are merged using a randomized algorithm [9]. In the episode, Bender and the office soda machine Bev mate and produce a child together that morphologically and behaviorally shares traits from each parent. Their son, Ben, shares the same color as his father, but the torso shape of his mother. Important to the plot of the episode is Ben's inability to bend girders, much to his father's dismay, as this was an innate skill that Bender was built with.

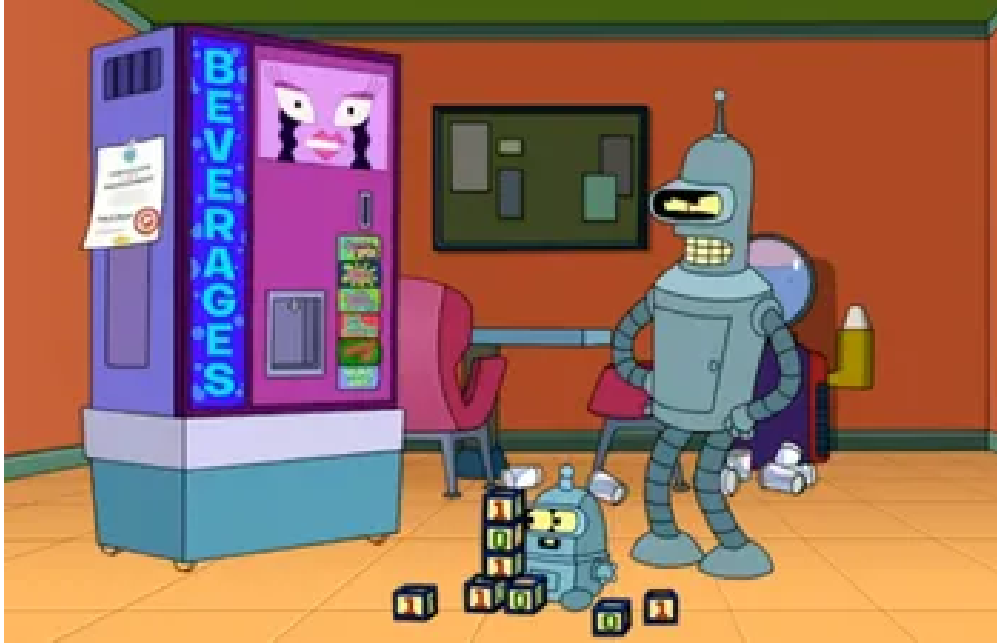


Figure 5: Bender’s robotic family. From left to right, Bev, Ben, and Bender.

This inheritance pattern supports the reproduction model presented in LambdaForge. Given that robotic reproduction is a stochastic recombination of features originating from parents, the sub-tree model of reproduction used in LambdaForge conforms to this concept portrayed in *Futurama*. In the show, perhaps each sub-tree from the binary files that dictate the characteristics of the robot from each parent undergo a similar process where the basic logical processes that define the robot mirror the genetic material that is involved in organic reproduction. This sub-tree-thinking of logic would allow for the conservation of properties passed down from parent to offspring, rather than a jumbling of rudimentary logic that would lead to potential incompatibilities.

This example from *Futurama* represents a potential development using the Lambda calculus framework for algorithmic reproduction. For example, having such large-scale features such as Ben’s retention of Bender’s body color conserve in reproduction. A potential explanation for this would be a heuristic mechanism that allows for only the recombination of a certain sub-tree size such that the logic used in the reproductive process contains useful information rather than potentially meaningless Lambda expressions derived from the

greater Lambda expression: the expression that defines the robotic algorithm in a manner analogous to the genetic information needed for organic reproduction. By having a method to conserve features across reproduction, inheritance is enabled as a concept within robotic reproduction. As inheritance is one of the necessary components for evolution to occur, an analysis of potential situations that would enable evolution is called for [6].

4.2 Evolution

In the episode "A Clockwork Origin" (Season 6, Episode 9), Professor Farnsworth flees Earth after giving up on humanity for their abandoning of the concept of the evolution, where they comically celebrate his finding of a new hominid *Homo farnsworth* by portraying it riding a dinosaur at "the moment of creation" [8]. After leaving Earth for his new planet, the Professor seeds a local pond with "nanobots" to decontaminate the water [8]. Upon waking up the next morning to check on the progress of the robots, the Professor is shocked to see the bots have evolved rapidly overnight to the robotic equivalent of the Jurassic age. While inheritance has been derived from the LambdaForge model for robotic reproduction, the topics of interest that would allow for evolution like this to occur robotically are the concepts of variation and selection.

The nanobots initially crafted by the Professor appear to all be identical, yet they exhibit drastic morphologic and behavioral divergence as they undergo evolution. A potential answer is hinted at in the mechanisms underlying the reproduction method used in LambdaForge. Given that reproductive events in LambdaForge are stochastic in their selection of logic sub-tree, a population of identical robots would have variation introduced in their next generation merely through the process of reproduction. In this scenario, reproduction itself is a cause of variation. Where this differs from organic reproduction is that the source of variation is not by means of genetic shuffling, where new combinations of genes and genetic elements serve to add variation, but rather analogous to creating *de novo* genes through the process of reproduction itself [5]. In this framework, offspring generations of

robots would contain new genes as a source of variation. This would allow for an identical population to create variation on its own, without the need for existing variation. But, this variation requires a means of performance scoring in order for selection to act on the population [2].

To answer the question of selection, a system is required such that the concept of fitness can be determined. Organically, individual fitness is defined by one's ability to survive and reproduce, passing their genes onto the next fertile generation. In the case of nanobots, a potential answer to the question of fitness is one that mirrors the organic. Evidence for this is in the convergence of nanobot evolution with organic evolution seen on Earth. The initial nanobots evolve into a radiation of mechanical beings that mirror the morphology and behavior of extant species on Earth. Robots that resemble species such as humans, birds, and insects are shown to have evolved in the robotic system. An example of such is the robotic humanoids which are seen to have undergone a similar evolutionary history as humans, appearing to have evolved from ancient primate robots.



Figure 6: The comical evolutionary path of robotic humanoids seen in the nanobot system.

4.2.1 The Issue of Scoring Fitness

Yet, the utility of robots is not necessarily in their ability to reproduce like organic life. Rather, robots are graded on their ability to perform specific tasks that define their utility. Thus, determining the fitness of a robot must include a testing and analysis of a robot's performance of their specific task such that following generations are the byproducts of expressions with successful logic. I hypothesize that testing the abilities of evolving logic requires different approaches dependent on the difficulty or subjectivity of the performance metric. For simple processes with directly comparable metrics, the output metrics can be compared such that expressions with desired metrics will continue in the next generation. But, with more subjective or sophisticated metrics, I predict that may need be the incorporation of artificial intelligence architectures. Given this, scoring would have to arrive from specific embeddings that describe the behavior of Lambda expressions such that more complex outputs can be determined.

5 Case Study: Finding a Novel Function using LambdaForge

Using LambdaForge, proof-of-concept novelty can be explored. A case of such is the emergence of the successor function from the reproduction of the addition and multiplication operators. Given the input expressions of the two operators, LambdaForge formed the Lambda expression $\lambda x.f(nfx)$, the Lambda expression equivalent for the successor function. This novel functionality becomes clear when the abstract functions that define multiplication and addition are considered. Addition itself is a loop of succession, where input values undergo succession a set number of times. Multiplication is identical logic where input values undergo addition of itself a set number of times. Both of these functions themselves are built and reliant on the functionality of the successor function. Thus, by returning the successor function in the reproduction of addition and multiplication, it is apparent that LambdaForge is able to not only return meaningful, unique functions from using the logic

of parent functions, but also maintain the logic that defines the parent functions in the following generations.

6 Implications and Future Directions

While LambdaForge currently operates as a conceptual framework for simulating Lambda calculus expressions, the underlying architecture is readily extensible to produce more advanced evolutionary scenarios. For example, developing mechanisms to gauge fitness of child expressions and adding heuristics to the reproductive process to maintain important sub-trees from parent functions would allow for more sophisticated recursive recombinatorial reproduction that would mirror evolution. With sufficient computational resources, reproduction could be scaled from simple lambda expressions to real-life logical processes.

With continued human reliance on technology, reality has begun to mirror the science fiction works produced in past. From the dawn of the industrial revolution, technology has shaped and developed society at an accelerating rate. Beginning with the evolution of manual labor seen through such technologies as automation, intellectual labor has begun to be aided at greater rates with each coming year due to the intervention of artificial intelligence. I anticipate that this reliance on technology will only continue to increase. As shown in *Futurama*, offering robots both the ability to reproduce and evolve would reduce the manual and intellectual labor required to pursue production and functionality development. Looking ahead, such technology will require careful and comprehensive regulation.

References

- [1] BEESON, M. The church numbers in nf set theory, 2021. 92 pages, 2 figures. This version added Section 20; the previous Section 20 became Section 21.

- [2] BLACKISTON, D., LEDERER, E., KRIEGMAN, S., GARNIER, S., AND LEVIN, M. A cellular platform for the development of synthetic living machines. *Science Robotics* 6, 52 (March 2021). ORCID: 0000-0001-7231-0196, 0000-0003-0178-409X, 0000-0002-9207-1775, 0000-0002-3886-3974, 0000-0001-7292-8084.
- [3] CHODASEWICZ, K. Evolution, reproduction and definition of life. *Theory in Biosciences* 133, 1 (May 2013), 39–45.
- [4] CHURCH, A. An unsolvable problem of elementary number theory. *American Journal of Mathematics* 58, 2 (1936), 345–363.
- [5] CRAMERI, A., RAILLARD, S. A., BERMUDEZ, E., AND STEMMER, W. P. Dna shuffling of a family of genes from diverse species accelerates directed evolution. *Nature* 391, 6664 (January 1998), 288–291.
- [6] FORBES, A. A., AND KRIMMEL, B. A. Evolution is change in the inherited traits of a population through successive generations. *Nature Education Knowledge* 3, 10 (2010), 6. © 2010 Nature Education.
- [7] FU, C., COELHO, M. A., DAVID-PALMA, M., PRIEST, S. J., AND HEITMAN, J. Genetic and genomic evolution of sexual reproduction: Echoes from leca to the fungal kingdom. *Current Opinion in Genetics & Development* 58–59 (August 2019), 70–75. Author manuscript; available in PMC: 2020 Oct 1. Published in final edited form as: *Curr Opin Genet Dev*. 2019 Aug 29;58-59:70–75.
- [8] FUTURAMA WRITERS. A clockwork origin. <https://www.imdb.com/title/tt1704527/>, 2010. Season 6, Episode 9, aired August 12, 2010.
- [9] FUTURAMA WRITERS. The bots and the bees. <https://www.imdb.com/title/tt2304866/>, 2012. Season 7, Episode 1, aired June 20, 2012.

- [10] GROENING, M., AND COHEN, D. X. Futurama. Television series, 1999–2013. Created by Matt Groening and developed by David X. Cohen. Originally aired on Fox and Comedy Central.
- [11] KUBRICK, S., AND CLARKE, A. C. 2001: A space odyssey. Motion picture, 1968. Metro-Goldwyn-Mayer (MGM).
- [12] LUCAS, G., ET AL. Star wars film series. Film series, 1977–2019. Produced by Lucasfilm Ltd.; distributed by 20th Century Fox and Walt Disney Studios Motion Pictures.
- [13] MALUSARE, A., KOTHANDARAMAN, H., TAMBOLI, D., LANMAN, N. A., AND AGGARWAL, V. Understanding the natural language of dna using encoder–decoder foundation models with byte-level precision. *Bioinformatics Advances* 4, 1 (08 2024), vbae117.
- [14] OSAWA, H., MIYAMOTO, D., HASE, S., SAIJO, R., FUKUCHI, K., AND MIYAKE, Y. Visions of artificial intelligence and robots in science fiction: a computational analysis. *International Journal of Social Robotics* 14, 10 (July 2022), 2123–2133.
- [15] SZATHMÁRY, E. Why are there four letters in the genetic alphabet? *Nature Reviews Genetics* 4, 12 (December 2003), 995–1001.